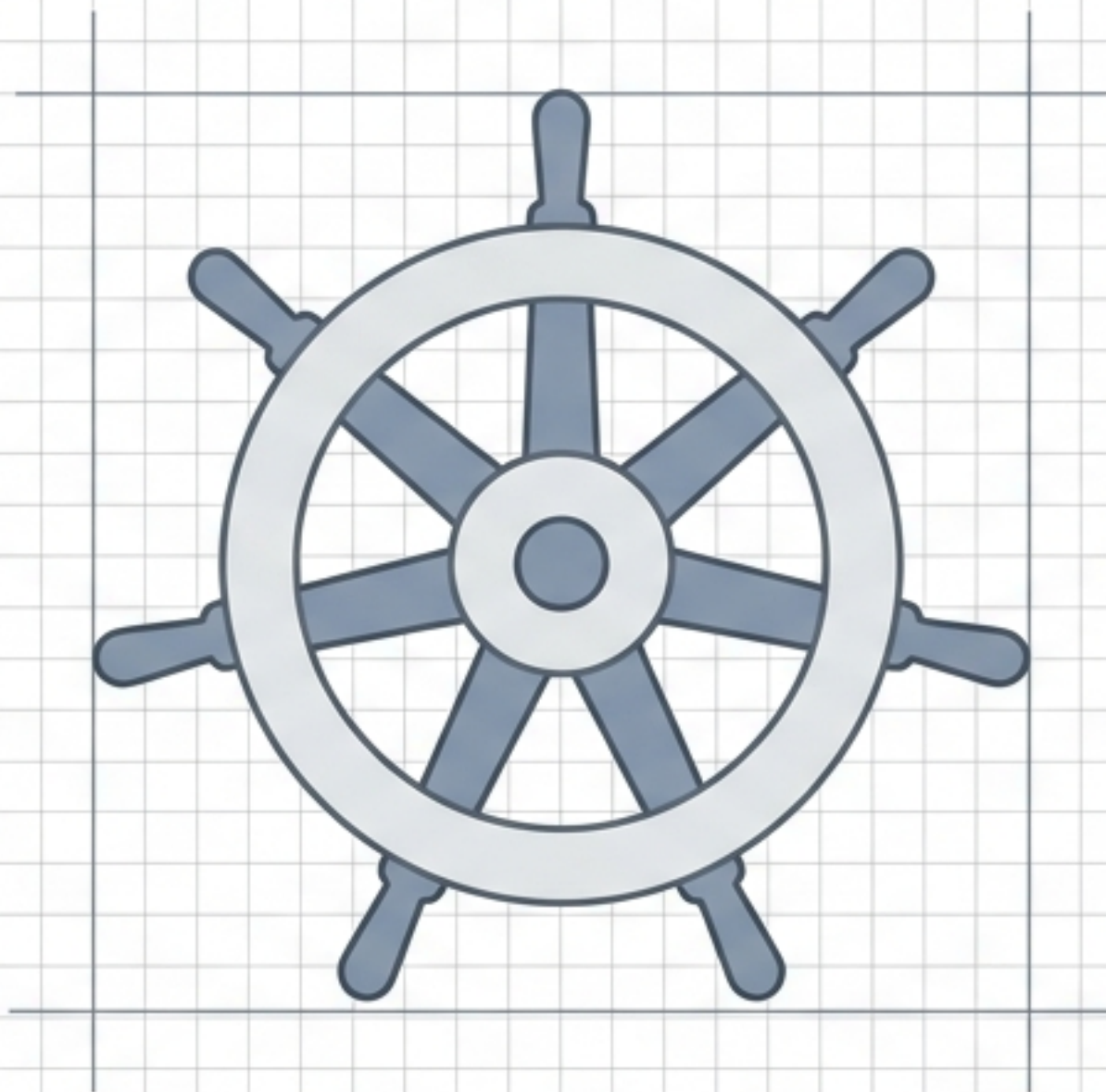




Kubernetes: Arquitetura e Fundamentos

Nodes, Control Plane, Pods, Deployments e Services.

Visão Geral do Sistema

Plataforma de código aberto para orquestração de contêineres em nível de produção.

1. Distribuição e Agendamento:

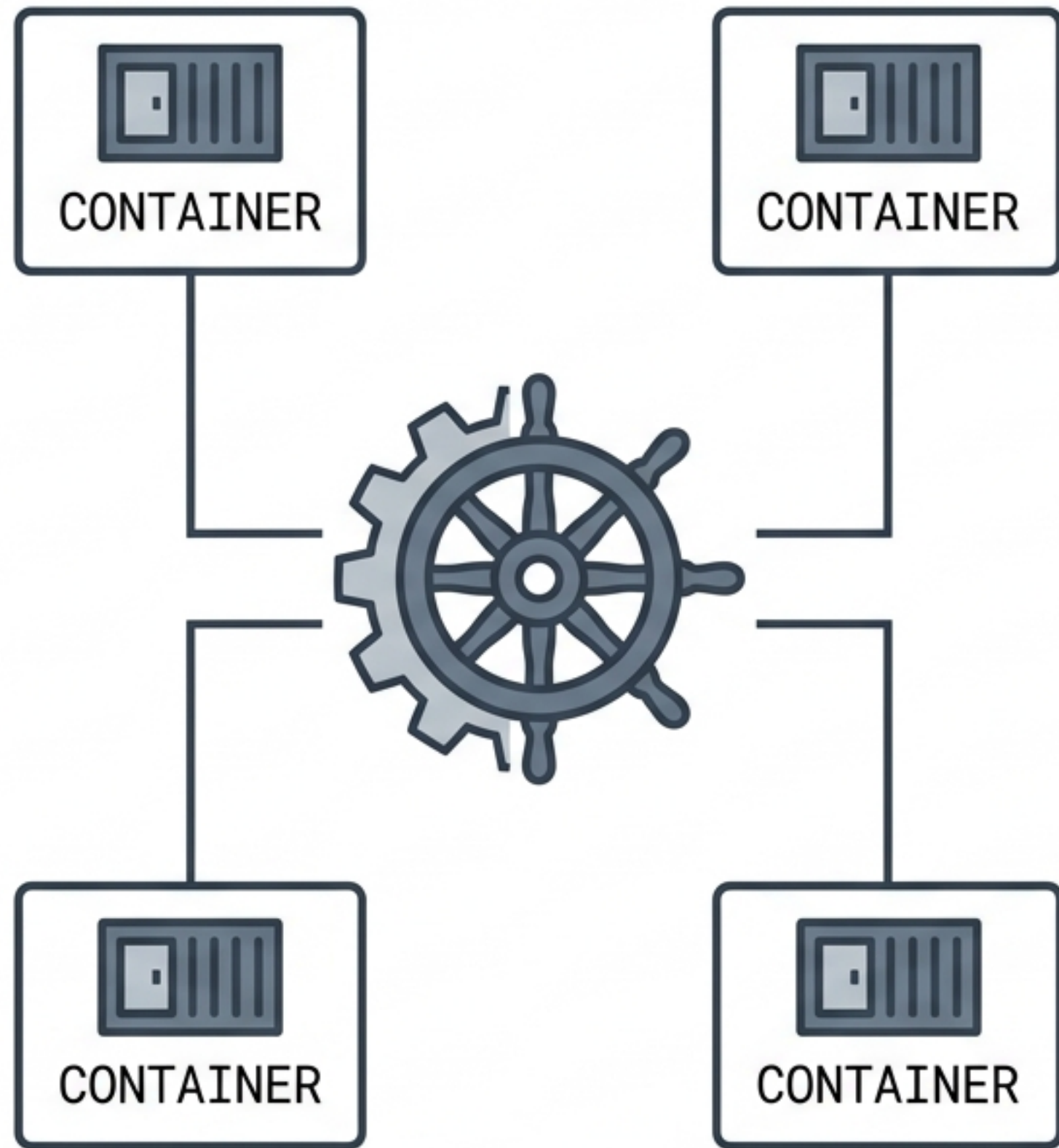
Automatiza a alocação e execução de contêineres em um cluster de computadores.

2. Desacoplamento:

Separa as aplicações da infraestrutura física subjacente.

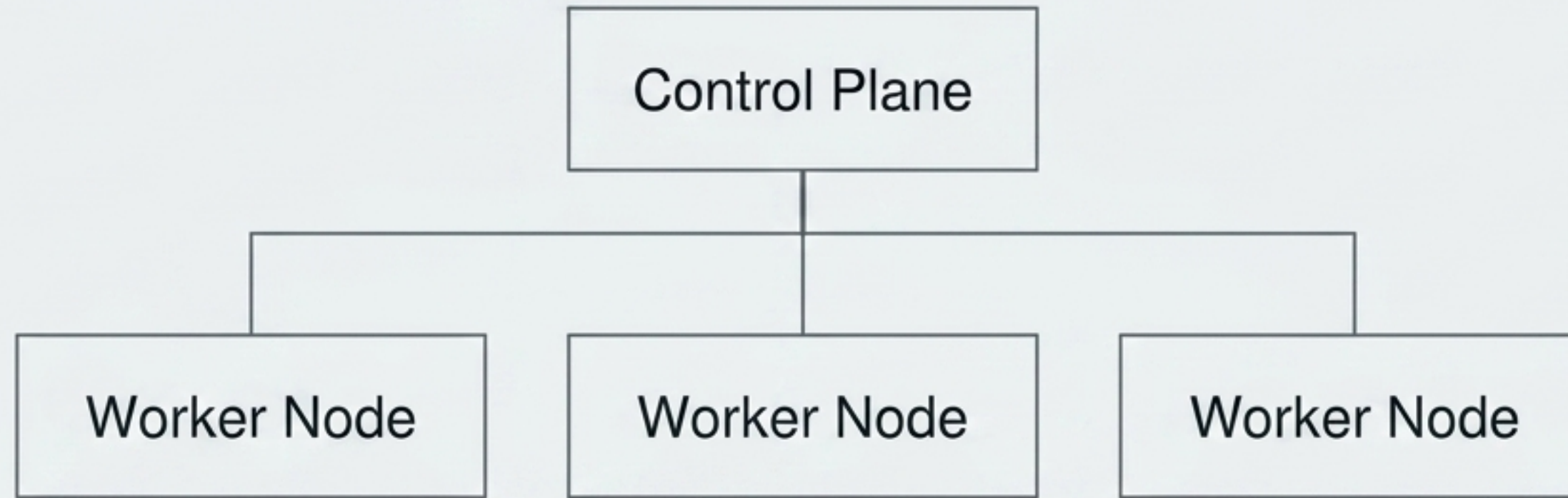
3. Autocorreção:

Fornece mecanismos contínuos para lidar com falhas de máquinas e manter o estado desejado da aplicação.



Arquitetura Macro: O Cluster

Um cluster Kubernetes consiste em dois tipos de recursos de infraestrutura:

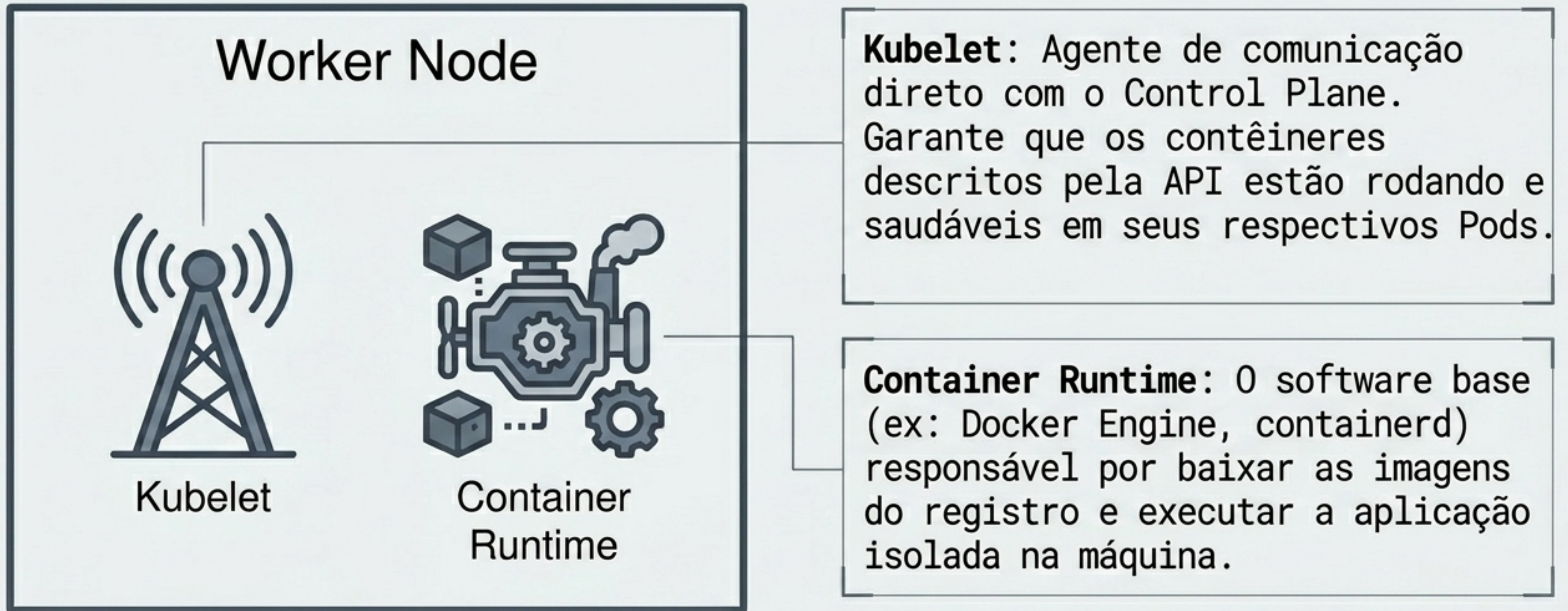


Camada de Gerenciamento (Control Plane):
Coordena todas as atividades do cluster.
Mantém o estado desejado das aplicações,
aloca cargas de trabalho e gerencia
atualizações sistêmicas.

Nós de Processamento (Nodes):
Máquinas físicas ou virtuais que hospedam e
executam as aplicações containerizadas.

Componentes Internos do Nó

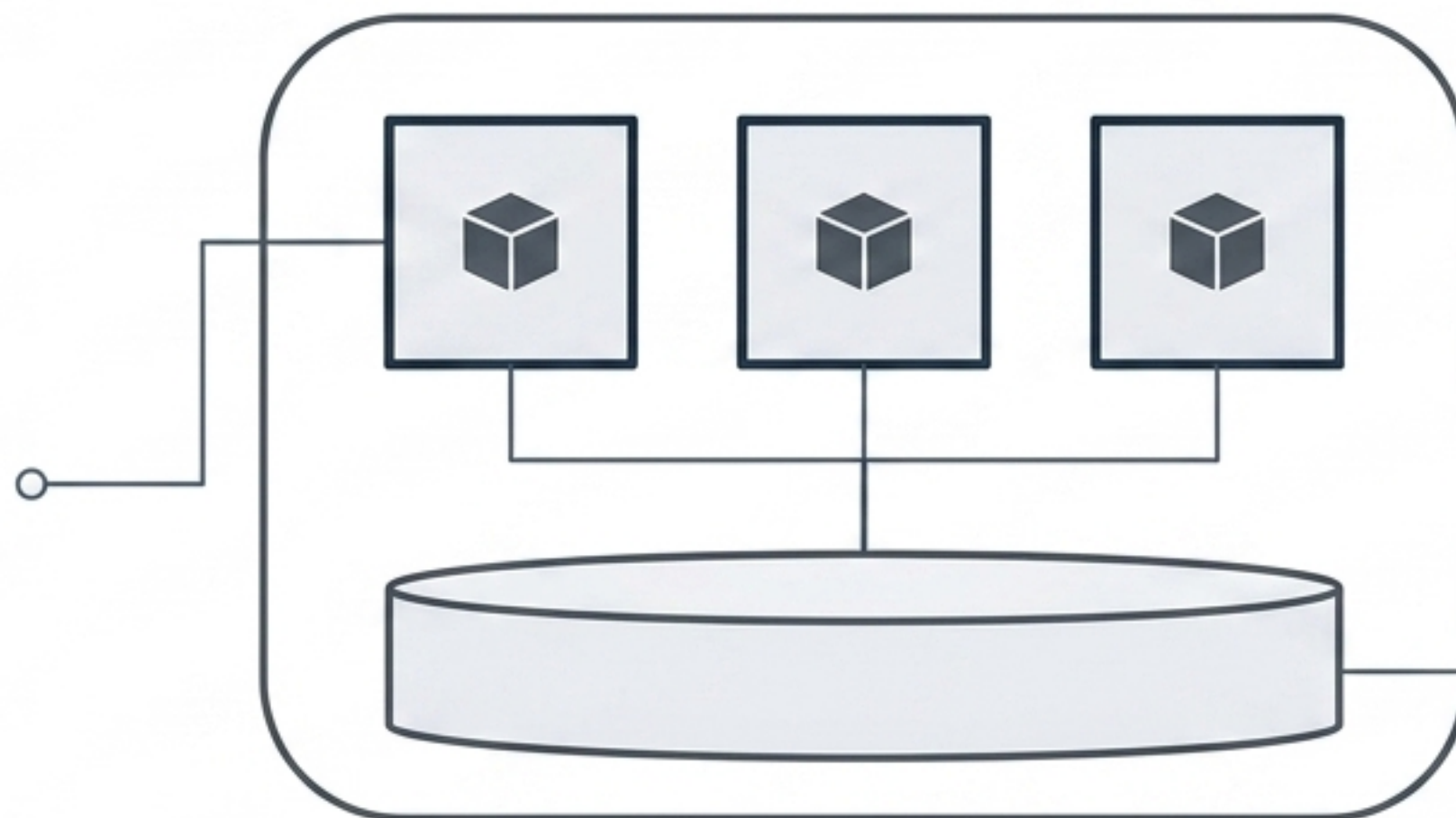
Cada Nó executa obrigatoriamente processos essenciais para hospedar aplicações:



Pods: A Unidade Atômica de Execução

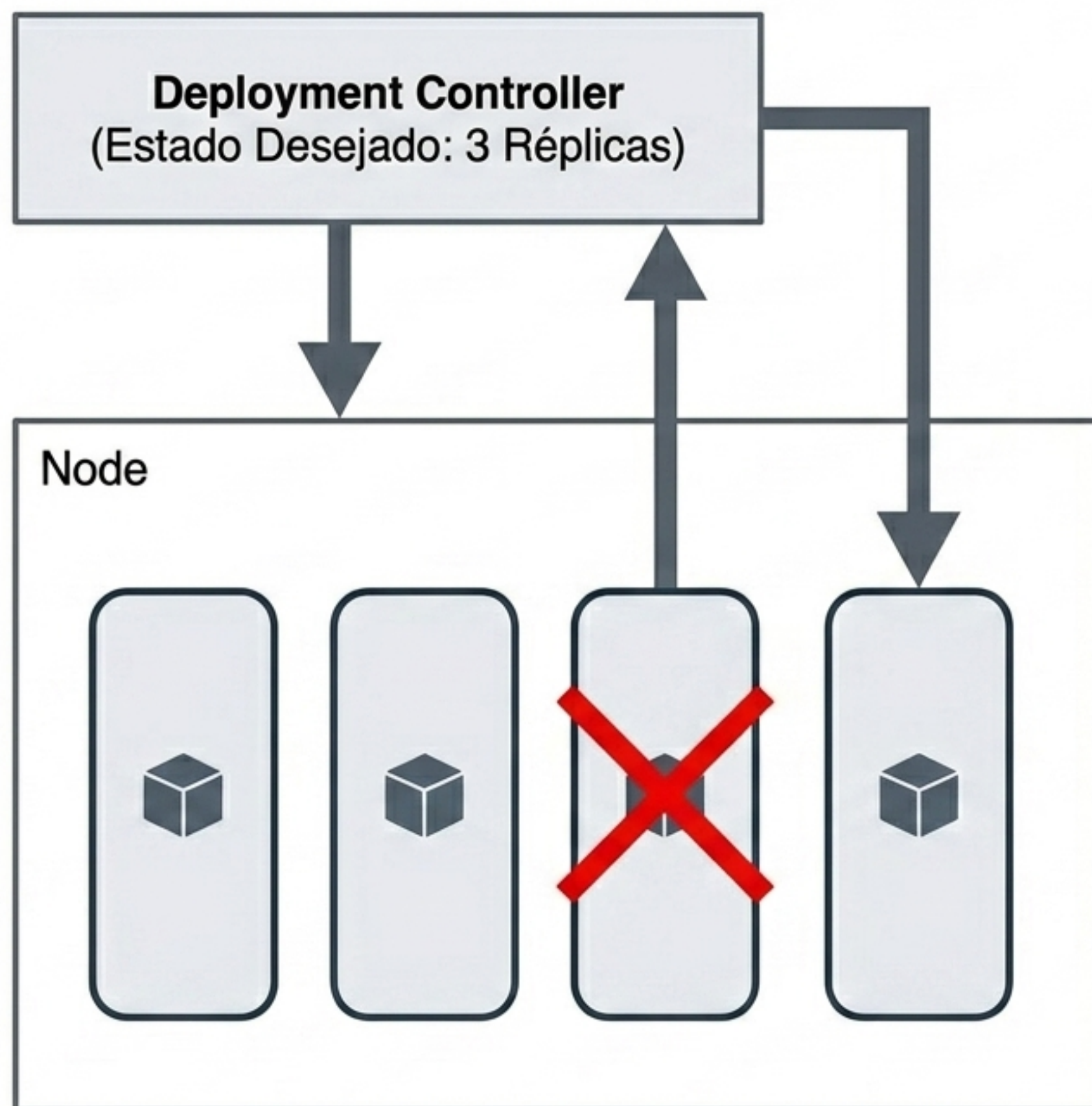
Abstração de um grupo de um ou mais contêineres fortemente acoplados. O Kubernetes gerencia Pods, não contêineres diretamente.

Contexto Compartilhado:
Contêineres no mesmo Pod compartilham o mesmo endereço IP, espaço de rede e intervalo de portas.



Armazenamento:
Compartilham acesso aos mesmos volumes físicos ou lógicos.

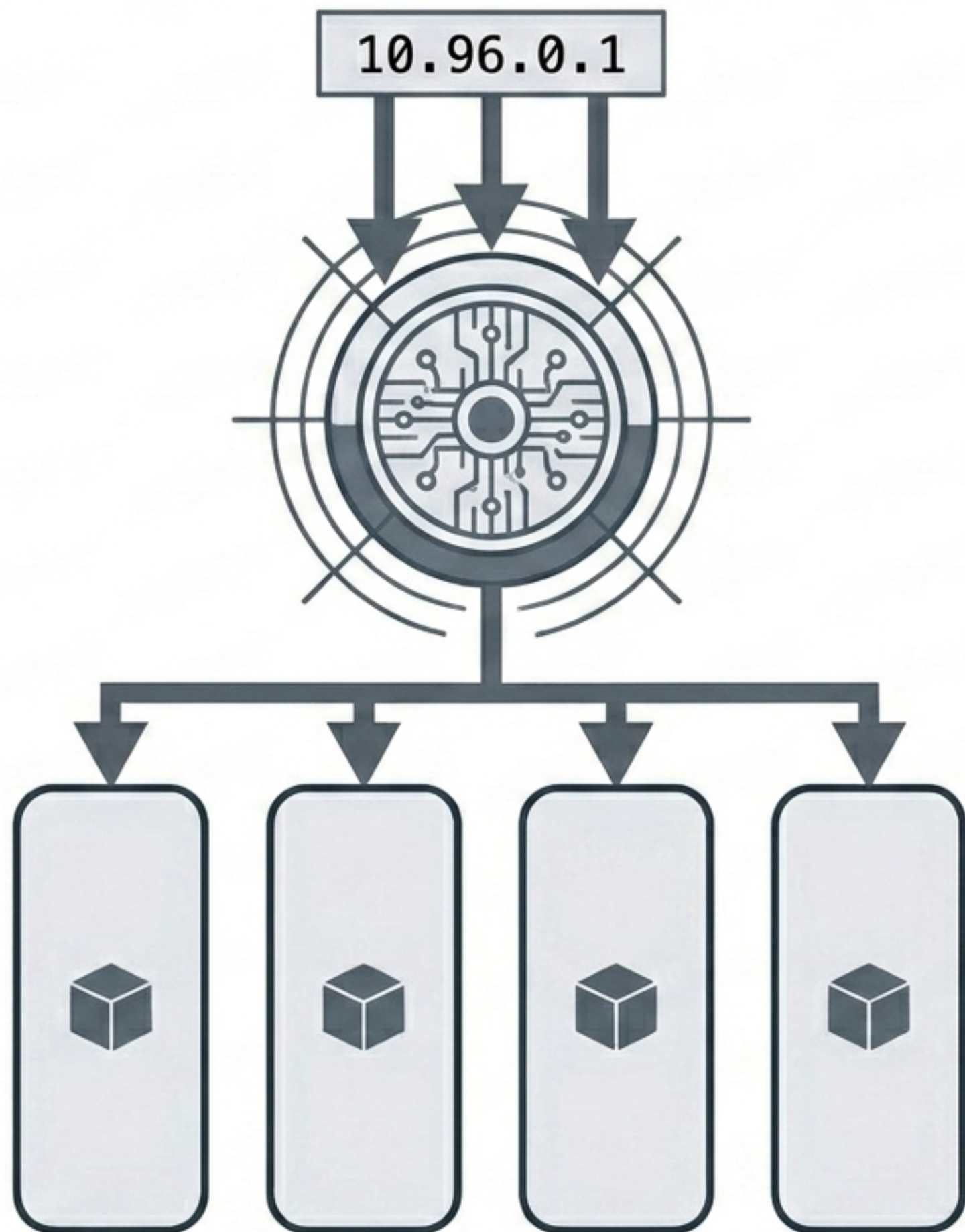
Efemeridade: Pods são mortais. Em caso de falha do Nó, o Pod é destruído e um novo substituto é recriado em outro Nó disponível.



Deployments: Gerenciamento de Estado

Controlador da Camada de Gerenciamento responsável por manter o estado desejado das aplicações.

Definição:	Especifica a imagem do contêiner e o número exato de réplicas requeridas.
Autocorreção:	Substitui automaticamente instâncias de Pods perdidas devido a falhas no Nó ou exclusão manual.
Atualizações (Rolling Updates):	Permite atualizações graduais de versão da aplicação sem tempo de inatividade (zero downtime).



Services: Roteamento e Conectividade

Abstração lógica que resolve a volatilidade dos endereços IP dos Pods.

- **Estabilidade:** Atribui um IP e uma porta estáticos e permanentes para um grupo de Pods.
- **Balanceamento de Carga:** Distribui ativamente o tráfego de rede de forma equilibrada entre todas as réplicas disponíveis.
- **Descoberta:** Monitora os endpoints continuamente, garantindo que o tráfego seja roteado apenas para Pods operacionais e saudáveis.

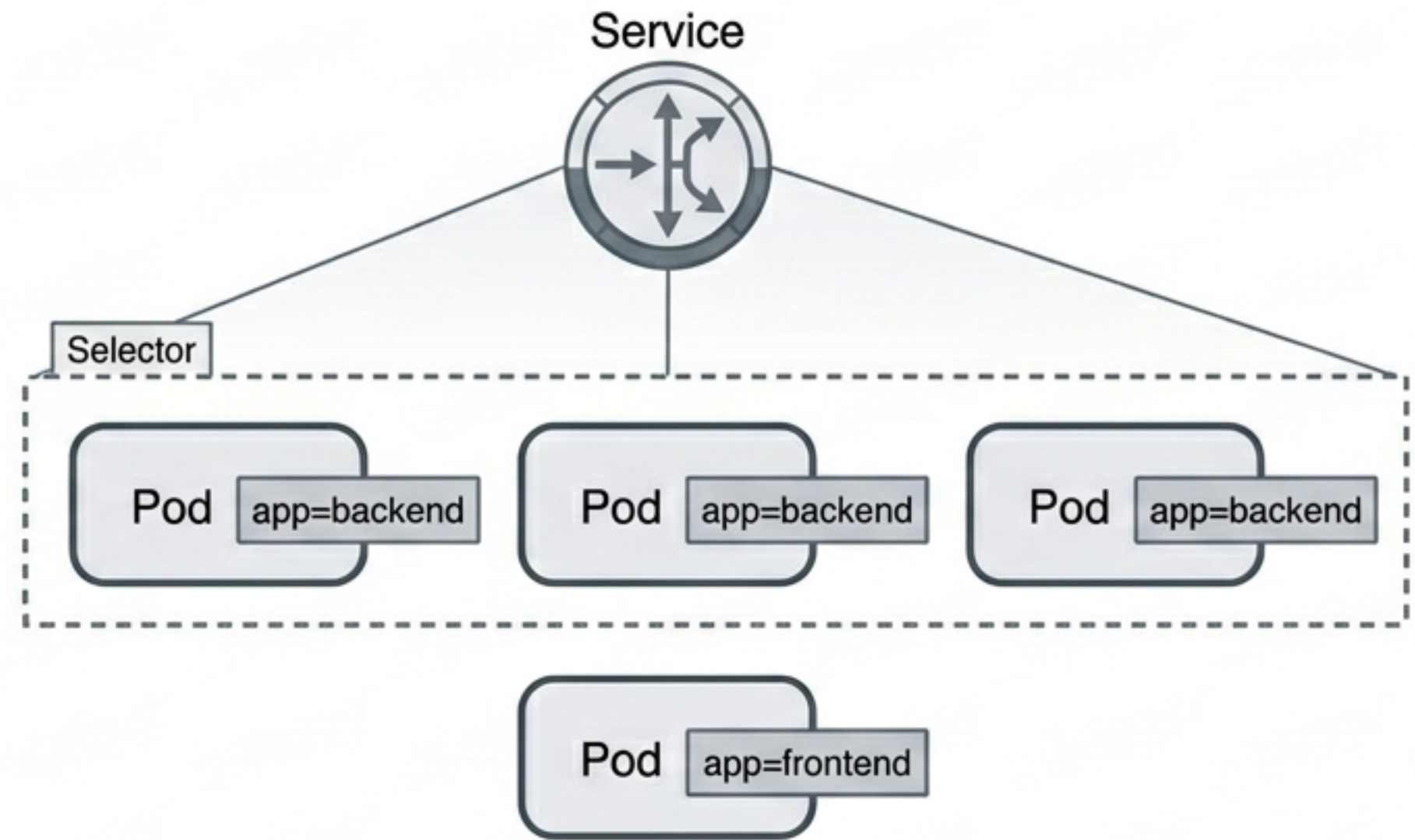
Matriz de Exposição: Tipos de Services

	ClusterIP	NodePort	LoadBalancer
Acessibilidade	Acesso estrito apenas ao interior do cluster.	Acessível externamente via IP do Nó.	Acessível via IP público definitivo.
Mecanismo	Cria um IP virtual interno (Padrão).	Abre uma porta estática fixa (ex: 30080) em todos os Nós de processamento (<NodeIP>:<NodePort>).	Provisiona um balanceador de carga externo diretamente no provedor de nuvem subjacente.
Caso de Uso	Comunicação entre microsserviços internos (ex: Backend -> Banco de Dados).	Acesso de desenvolvimento ou integração local.	Exposição de aplicações em ambiente de produção para o tráfego da internet.

Rótulos e Seletores (Labels & Selectors)

```
# No Pod:  
metadata:  
  labels:  
    app: backend  
    version: v1
```

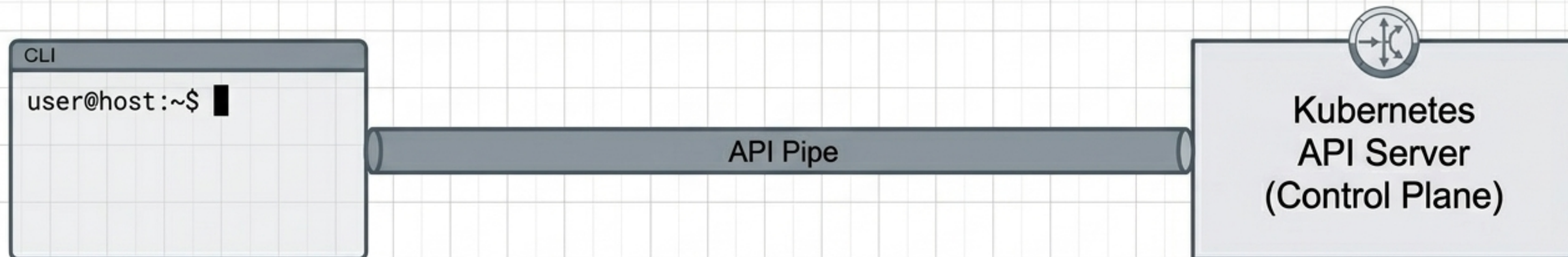
```
# No Service:  
spec:  
  selector:  
    app: backend
```



Rótulos: Pares chave/valor estáticos anexados a objetos para identificação e classificação lógica.

Seletores: Mecanismo de busca utilizado pelo Service (ou Deployment) para identificar dinamicamente quais Pods pertencem ao seu escopo de atuação.

A Interface de Linha de Comando: kubectl



Ferramenta principal para interagir e enviar instruções para a API do Kubernetes.

```
kubectl <ação> <recurso> <nome> [flags]
```

- **<ação>**: A operação desejada (ex: `create`, `get`, `describe`, `delete`, `logs`, `exec`).
- **<recurso>**: O tipo de objeto alvo (ex: `pods`, `deployments`, `services`, `nodes`).
- **<nome>**: O identificador único do recurso específico.

Comandos Práticos: Criação e Escalonamento

Cria um objeto **Deployment** instruindo o uso de uma imagem específica de contêiner.

```
kubectl create deployment meu-app --image=nginx:latest
```

Cria um **Service** para o **Deployment**, mapeando uma porta acessível externamente.

```
kubectl expose deployment nginx --type=NodePort --port=80 --target-port=80
```

Ajusta o número de **réplicas** desejadas para gerenciar aumento de tráfego.

```
kubectl scale deployment/meu-app --replicas=4
```

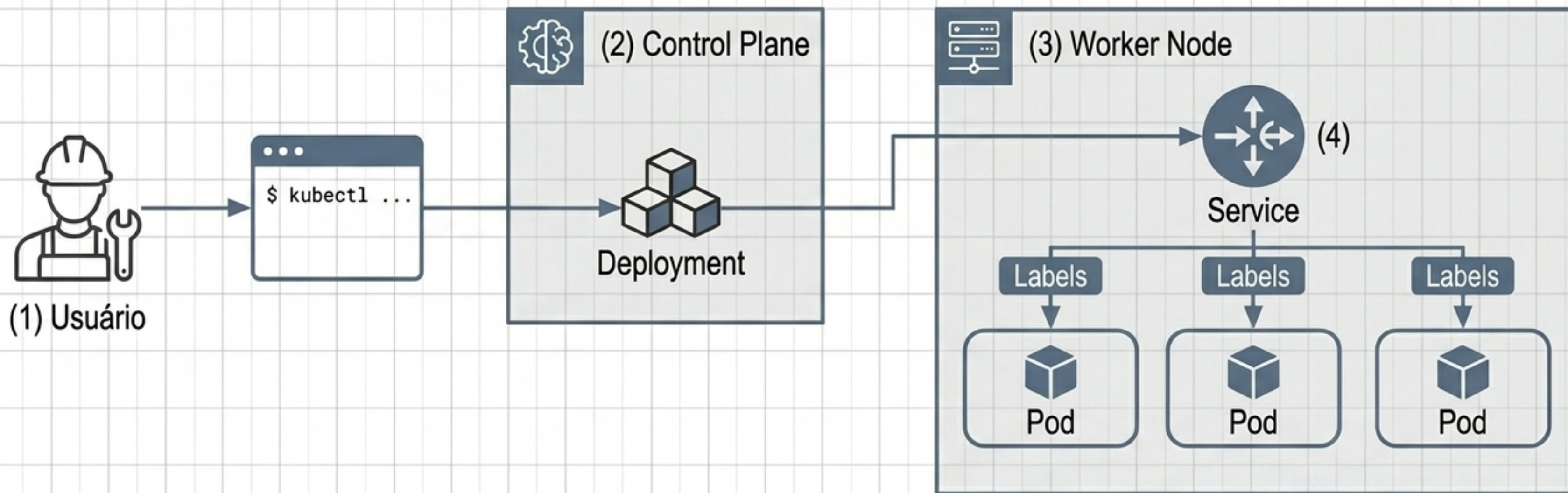
Encaminha uma porta local para uma porta do serviço.

```
kubectl port-forward services/nginx 8080:80
```


Comandos Práticos: Diagnóstico e Depuração

Listar recursos e exibir IPs.	Exibir log de eventos e propriedades detalhadas de um objeto.
<code>kubectl get pods -o wide</code> user@host:~\$	<code>kubectl describe pod</code> <nome-do-pod>
Acessar a saída padrão (stdout) emitida pelo contêiner.	Abrir uma sessão interativa (shell) diretamente no contêiner em execução.
<code>kubectl logs <nome-do-pod></code>	<code>kubectl exec -it <nome-do-pod></code> -- bash

Síntese: Fluxo de Execução Kubernetes



1. Instrução: O desenvolvedor declara o estado desejado utilizando `kubectl`, enviando a requisição para a API do Control Plane.

2. Gerenciamento: O controlador de Deployment processa a requisição e cria as réplicas lógicas.

3. Alocação: O Kubelet no Nó de processamento recebe a ordem e aciona o runtime para baixar as imagens e subir os Pods.

4. Acesso: Um Service atua como balanceador estático, roteando o tráfego externo para os Pods adequados utilizando Rótulos e Seletores.